

索引 (以空间代价换取时间效率提升)

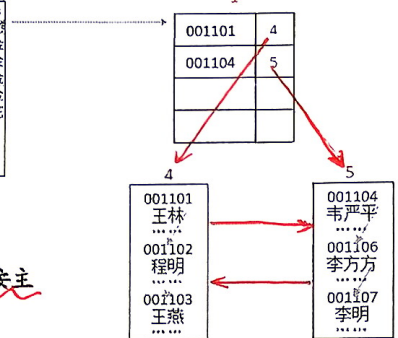
数据库索引是数据库管理系统中一个排序的数据结构, 协助快速查询、更新数据库表中的数据

B+树索引 { 聚集索引
非聚集索引 (二级索引 / 辅助索引)

B+树的非叶子节点只能存储键值对, 实际的数据记录都存储在叶子节点中
B+树的叶子节点按主键值顺序排列, 同层节点之间都有双向指针, 相邻节点有顺序引用关系。

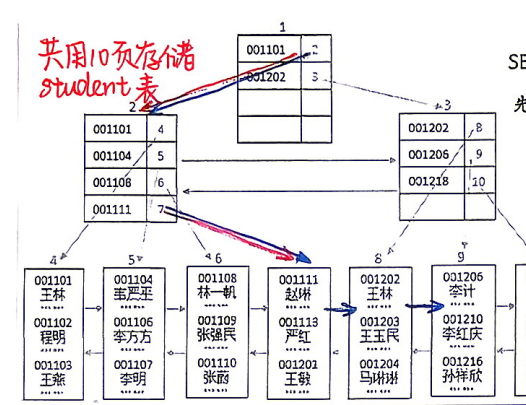
聚集索引 将数据存储与索引放在一起, 表中行的逻辑顺序与主键值的索引顺序相同
一张表只有一个聚集索引, 找到索引即找到行数据

sno	sname	dept	sex	birthday	totalcredit	remarks
001101	王林	计算机	男	2000-01-01	50.0	三好学生
001102	程明	计算机	男	2000-02-01	50.0	获奖学金
001103	王燕	计算机	女	2000-01-03	50.0	获奖学金
001104	韦严平	计算机	男	2000-01-04	50.0	获奖学金
001106	李方方	计算机	男	2001-01-05	50.0	获奖学金
001107	李明	计算机	男	2003-06-06	64.0	提前修完



一页中存储的键越多
↓
扇出越多 (子节点指针数量)
↓
存储非叶子节点所需页数越少
↓
树高度越低
↓
查询速度越快

- 创建表时, 系统为表分配一个16K大小的根页
- 插入表中的记录最初存储在根页中, 页内记录按主键从小到大存放, 并建立单链表。
- 当根页中空间不足时, 系统会再分配两个空的数据页, 将根页中的数据拷贝到数据页中, 根页变为目录页, 目录页中存放主键值和页指针, 并建立页间指针。



例1: 查询学号为 '001111' 的学生。
SELECT * FROM student WHERE sno='001111';
先后调入内存的页: 根页1 → 目录页2 → 数据页7
要读3页数据 (3次IO操作)

例2: 查询学号大于 '001111' 且小于 '001206' 的所有学生。
SELECT * FROM student WHERE sno>'001111' AND sno<'001206';
先后调入内存的页: 根页1 → 目录页2 → 数据页7 → 数据页8 → 数据页9
找完本页还未完成, 就利用页间指针将下页调入内存, 页内搜索直至不满足查询条件

创建聚集索引 (主键自动作为聚集索引)

① 建表时定义:

```
CREATE TABLE stu (  
Sno CHAR(6) PRIMARY KEY, ...);  
或  
CREATE TABLE stu (  
Sno CHAR(6), ...  
PRIMARY KEY (sno));
```

② 修改表时创建:

```
ALTER TABLE stu  
ADD PRIMARY KEY (sno);
```

若不定义主键, 系统将使用第一个唯一 (UNIQUE) 索引
创建聚集索引, 要是没有就自动生成 rowid 作隐藏的聚集索引主键

使用 EXPLAIN 关键字可以模拟优化器执行SQL语句, 分析你的查询语句

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student		const	PRIMARY	PRIMARY	24	const	1	100.00	Using where

EXPLAIN SELECT * FROM student WHERE sno='001111'; 用聚集索引效率高
主键值与常量比较 符合的占比

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student		ALL	(NULL)	(NULL)	(NULL)	(NULL)	21	10.00	Using where

EXPLAIN SELECT * FROM student WHERE sname='王林'; 全表扫描 符合的占比

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student		range	PRIMARY	PRIMARY	24	(NULL)	6	100.00	Using where

EXPLAIN SELECT * FROM student WHERE sno>'001111' AND sno<'001206'; 范围查询 (主键上)

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student		ALL	(NULL)	(NULL)	(NULL)	(NULL)	21	23.33	Using where

EXPLAIN SELECT * FROM student WHERE totalcredit> 45; 全表扫描

聚集索引 + WHERE 条件中使用主键值
查询效率高

辅助索引 (二级索引) 每列表有多个字段, 可创建多个二级索引

一般在非主键字段上创建, 例如用 index 创建的一般索引或用 unique index 创建的唯-索引

创建二级索引:

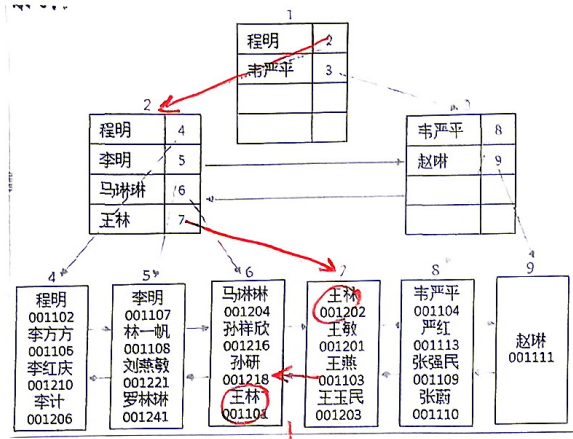
① CREATE TABLE student1

INDEX idx-sname (sname)
) CHARSET = gbk;

② CREATE INDEX idx-sname
ON student (sname);

作书签, 告诉存储引擎去那找与索引对应的行数据

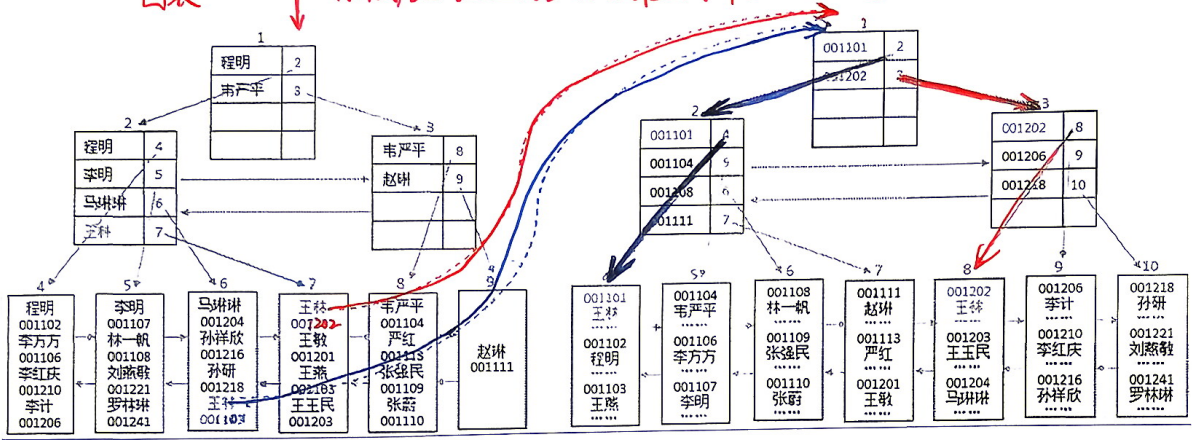
- 数据页中按拼音顺序存储索引键值, 并存储相应主键值
- 目录页中存储索引键值和页指针



欲知详细信息, 回表

先利用二级索引找到相应的主键值, 再根据主键值用聚簇索引查询详细信息 (行数据)

执行效率低



字段 sname 上已创建索引

这里有覆盖索引, sno 就是主键, 二级索引中有, 不用回表查完整行数据

SELECT sno FROM student WHERE sname = '王林';

filtered 都是 100

SELECT * FROM student WHERE sname = '王林';

要回表, 执行效率不如上面的

联合索引 (多列索引, 键的数量不止一个) 也是二级索引!

联合索引也称多列索引, 是建立在多个字段上的索引;

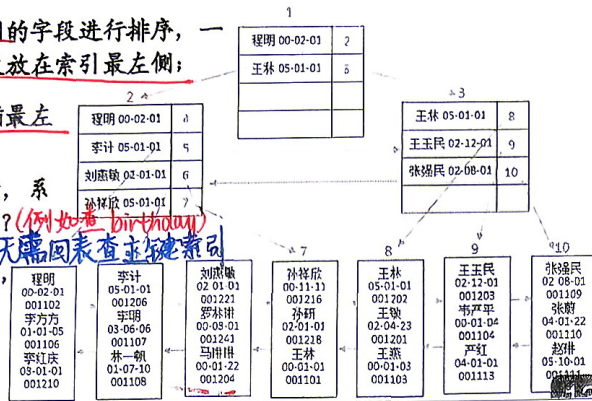
联合索引中的数据根据最左侧的字段进行排序, 一般把查询中最频繁使用的字段放在索引最左侧;

利用联合索引进行查询要遵循最左前缀原则;

有时查询的字段不是最左侧的, 系统也使用索引来查询; 为什么? (例如 birthday)

索引覆盖, 效率比聚集索引高, 无需回表查主键索引

Covering Index



每页中存放的键值对会减少

树的高度会增高

查询性能会降低

(但还是比聚集索引要快的)

创建联合索引:

```
CREATE INDEX idx_sname-birthday  
ON student (sname, birthday);
```

假设按主键 sno 字段建立聚集索引名为 primary,

按 sname 字段建立辅助索引名为 idx_sname,

按 sname 字段和 birthday 字段建立联合索引名为 idx_sname-birthday

使用辅助索引 ↓ 和聚集索引

SELECT * FROM student WHERE sname = '王林';

使用辅助索引 ↓

SELECT sno FROM student WHERE sname = '王林';

使用联合索引 ↓ 和聚集索引

回表

SELECT * FROM student WHERE sname = '王林' AND birthday = '2005-01-01';

使用联合索引 ↓

不回表 SELECT sno FROM student WHERE sname = '王林' AND birthday = '2005-01-01';
(使用索引覆盖)

使用联合索引

SELECT sno, sname FROM student WHERE sname = '王林' AND birthday = '2005-01-01';

使用联合索引 ↓

B+ 树高度比聚集的低, 即使全表扫描也比在聚集索引中全表扫描高 filtered ≠ 100

SELECT sno, sname, birthday FROM student WHERE birthday = '2005-01-01';

不按这个排的 (按 sname), birthday 无序

不使用索引 ↓ 全表扫描 (ALL)

回表 SELECT sno, sname, birthday, dept FROM student WHERE birthday = '2005-01-01';
filtered ≠ 100

全文索引

由于B+树索引的最左前缀原则, 有些情况下即使添加B+树索引也要全表扫描

例如 `SELECT * FROM student WHERE remarks LIKE '%三好学生%';`

MySQL支持3种模式的全文索引

{	自然语言(默认)
	布尔
	查询扩展

只有 CHAR、VARCHAR、TEXT 等数据类型的字段才可创建全文索引

创建全文索引:

```
CREATE FULLTEXT INDEX ft-remarks  
ON student (remarks);
```

使用:

```
SELECT *  
FROM student  
WHERE MATCH (remarks) AGAINST ('三好学生');
```

Hash 索引

MySQL没有显示支持Hash索引而是采用自适应Hash索引

Hash索引自身只存储对应Hash值, 结构紧凑、查找速度快

→ 但每次查询时都要遍历Hash表直至找到对应Hash值, 数据量大时表也大

→ 只支持 "=" "IN" "<=>" 精确匹配索引所有列的查询, 不支持范围查询、部分索引列匹配查询

→ 当不同索引值算得相同Hash值时, 产生冲突,

索引的设计原则

① 命名规范

主键索引名 pk-字段名, 唯一索引名 uk-字段名, 普通索引名 idx-字段名

② 查询条件中使用频繁的字段、数字型的字段、存储空间较小的字段适合建立索引

如 WHERE 子句、GROUP BY 子句、ORDER BY 子句、表与表间连接运算等频繁使用的字段

③ 重复度较高的字段、更新频繁的字段不适合建立索引

④ 若索引字段值很长, 则尽量使用前缀建立索引

如何不确定取多少字符?

如
CREATE INDEX idx-cname
ON course (cname(6));

SELECT
COUNT(DISTINCT LEFT(字段名, 取位数))/
COUNT(字段名)
FROM 表名

由低往高去执行, 到结果为1时, 就取那个位数

⑤ 限制索引数量, 否则占用过多磁盘空间导致系统效率下降

索引的使用原则

① 查询时保证字段独立 (索引字段不能是表达式的一部分或函数的参数)

... WHERE id + 2 > 3 不使用索引, 全表扫描

... WHERE id > 3 - 2 使用索引

... WHERE sno + '2' > '001106' 不使用索引, 全表扫描

... WHERE sno > '001106' 使用索引
- '2'

② 模糊查询中通配符不要放在最左边使用, 否则会导致全表扫描而不使用设置的索引

例如 student 表中已按 sname 字段建立索引 sname-index, 那么

SELECT * FROM student WHERE sname LIKE '王%'; 使用

SELECT * FROM student WHERE sname LIKE '%王%'; 不使用

其实 MySQL 的优化器在查询时会判断全表扫描是否比索引慢, 要是更快还是会选全表扫描

③ 最左前缀匹配原则

对联合索引而言, 系统按从左到右的顺序使用索引中的字段, 一个查询仅可使用索引的最左侧部分

如 index(a, b, c) 支持使用 a、a, b、a, b, c 这三种组合, 而不支持 b, c 或 c.

若已建立联合索引 idx-sname-sex-birthday (CREATE INDEX idx-sname-sex-birthday ON student(sname, sex, birthday));

SELECT * FROM student WHERE sname = '王林' AND sex = '男'; 使用

SELECT * FROM student WHERE birthday = '2001-01-01' AND sname = '王林'; 使用

SELECT * FROM student WHERE sex = '男' AND birthday > '2001-01-01'; 不使用, 全表扫描.

④ 当 SELECT 查询语句涉及的字段包含在联合索引中, WHERE 语句无需满足最左前缀匹配原则, 系统也会按索引执行, 无需回表, 即索引覆盖.

SELECT sname FROM student WHERE birthday = '2001-01-01'; 使用

两字段都包含在联合索引中.

都要回表

创建索引

PRIMARY: 主键索引, 索引列值唯一且不能为空, 一张表只能有一个主键索引

INDEX: 普通索引

UNIQUE: 唯一索引, 索引列值唯一但可以为空

FULLTEXT: 全文索引

SPATIAL: 空间索引 (对空间数据类型的字段建立的索引)

Hash索引 (系统自建)

① 建表时创建索引

以 teaching-manage 中的 student 表为例, 创建对应索引:

在 sname 字段和 dept 字段创建联合索引 idx-sname-dept, 用于根据 sname 或 sname+dept 快速查询学生信息;

在 remarks 字段上创建全文索引 ft-remarks, 用于查询“三好学生”“学生干部”等信息。

```
CREATE TABLE student
(
  sno CHAR(6) PRIMARY KEY,
  sname VARCHAR(20) NOT NULL,
  dept VARCHAR(20),
  sex CHAR(1),
  birthday DATE,
  totalcredit DECIMAL(4,1) DEFAULT 0,
  remarks VARCHAR(100),
  INDEX idx-sname-dept (sname, dept),
  FULLTEXT INDEX ft-remarks (remarks)
);
```

② 建表后添加索引

在 course 表的 cname 字段创建唯一索引 uk-cname

```
CREATE UNIQUE INDEX uk-cname ON course(cname);
```

在 course 表的 cname 字段创建前缀索引 pf-cname

```
CREATE INDEX pf-cname ON course(cname(4));
```

CREATE INDEX 命令不能创建主索引!

③ 修改表结构添加索引

在 score 表的 sno 字段和 cno 字段创建联合唯一索引, 用来约束同一学生同一课程成绩仅出现一次 (不考虑补考, 否则违反唯一性约束)

```
ALTER TABLE score
ADD UNIQUE INDEX idx-sno-cno (sno, cno);
```

查看索引

① 查看索引

SHOW INDEX FROM 表名;

② 查看索引使用情况

查看 student 表中索引使用信息:

EXPLAIN SELECT * FROM student WHERE sname LIKE '王%';

查看 student 表中全文索引使用信息

EXPLAIN SELECT * FROM student WHERE
MATCH(remarks) AGAINST('三好学生');
 ↓ ↓
 列 关键字

删除索引

① 删除 student 表中的 idx-sname-dept 索引

ALTER TABLE student
DROP INDEX idx-sname-dept;

② 删除 score 表中的 uk-sno-cno 索引

DROP INDEX uk-sno-cno
ON score;

索引必须依表中列数据的使用特性创建。

使用索引可在查询过程中提升系统性能, 但会占用更多存储空间, 增、删都要重整 B+ 树结构, 拖慢速度。

视图

- 视图是从一个或几个基本表(或视图)导出的表,与基本表不同,是虚拟表
- 数据库中只存储视图的定义,视图的数据仍存放在基本表中
从单个基本表导出,去掉某些行列但保留主键
- 视图可分为(基于单表的视图 (行列子集视图) 4种,还有带表达式的视图(含虚拟列)、
基于多表的视图 分组视图(含聚合函数、GROUP BY子句)
基于视图的视图
基于表和视图的视图
- **视图的依赖性:**
视图依赖于视图定义中的基本表,若基本表被删除则它也随之失效.
但反之,视图的创建与删除只影响本身而不影响基本表
- 可将视图理解为存储起来的 SELECT 语句而非查询结果
- 视图在逻辑设计阶段建立
- 视图可看作一个窗口,反映一个表或若干表的局部数据,视图一经定义,就可将其当作表来查询数据
- 视图可定义在基本表或视图上,一个视图可在几个表或视图上建立,一个表或视图也可建立多个视图
- 优点:

- 操作简单
- 减少数据冗余
- 数据安全
- 适应灵活多变的需求
- 能够分解复杂的查询逻辑

视图的建立

① 创建基于单表的视图

view_dept 包含键, 不是行列子集视图; view_cs 和 view-ce 是行列子集视图

view-age 是带表达式的视图

view-avgscore 是分组视图

建立视图 view_dept, 包含系别信息

```
CREATE VIEW view_dept
```

AS

```
SELECT DISTINCT dept
```

```
FROM student;
```

建立计算机系的学生视图 view_cs, 包括 sno, sname, dept, birthday, totalcredit, remarks

```
CREATE VIEW view_cs
```

AS

```
SELECT sno, sname, dept, birthday, totalcredit, remarks
```

```
FROM student
```

```
WHERE dept = '计算机';
```

建立通信工程系学生的视图 view-ce, 包括 sno, sname, dept, sex, birthday, totalcredit, remarks, 并要求进行修改和插入操作后仍保证该视图只有通信工程系的学生

```
CREATE VIEW view-ce
```

AS

```
SELECT sno, sname, dept, sex, birthday, totalcredit, remarks
```

```
FROM student
```

```
WHERE dept = '通信工程';
```

```
WITH CHECK OPTION;
```

让 RDBMS 在该视图操作时自动加上 WHERE ... 条件

有这句话, INSERT INTO view-ce (sno, sname, dept) VALUES ('111', 'abc', '计算机'); 就不可执行了; 没有的话还是可以的

建立反映学生年龄的视图 view-age, 包括 sno, sname, age

```
CREATE VIEW view-age (sno, sname, age) → 写一处即可 (别名)
```

AS

```
SELECT sno, sname, YEAR(CURDATE()) - YEAR(birthday) AS age
```

```
FROM student;
```

INSERT INTO view-age VALUES ('111', 'abc', 20); 无法执行, 因为别名 age 的列由计算得到

建立反映学生平均成绩的视图 view-avgscore, 包括 sno, avg-score

```
CREATE VIEW view-avgscore (sno, avg-score)
```

别名

AS

```
SELECT sno, AVG(grade)
```

```
FROM score
```

聚合函数

```
GROUP BY sno;
```

```
UPDATE view-avgscore
```

```
SET avg-score = avg-score + 10
```

```
WHERE sno = '111';
```

无法执行, 数据库无法确定如何更新基本表数据来达到该效果

创建基于多表的视图

建立计算机选修了102号课程的学生视图 view-cs-102, 包括 sno、sname、grade

```
CREATE VIEW view-cs-102
```

```
AS
```

```
SELECT student.sno, sname, grade
```

```
FROM student, score
```

```
WHERE student.sno = score.sno AND dept='计算机' AND cno='102';
```

```
CREATE VIEW view-cs-102
```

```
AS
```

```
SELECT student.sno, student.sname, score.grade
```

```
FROM student JOIN score ON student.sno = score.sno
```

```
WHERE dept='计算机' AND cno='102';
```

创建基于视图的视图

建立计算机系选修102号课程且成绩不低于90分的学生视图 view-cs-102-90, 包含 sno、sname、grade

```
CREATE VIEW view-cs-102-90
```

```
AS
```

```
SELECT sno, sname, grade
```

```
FROM view-cs-102
```

```
WHERE grade >= 90;
```

视图的查看(必有 SHOW VIEW 权限)

① 查看视图的字段信息

查看 view-cs 的字段信息

```
DESCRIBE view-cs;
```

② 查看视图的创建语句(包含字符编码等)

查看 view-ce 的创建语句

```
SHOW CREATE VIEW view-ce;
```

③ 查看 information-schema 数据库下的 views 表

查看 view-cs-102 的信息

```
SELECT * FROM information-schema.views  
WHERE table-name = 'view-cs-102';
```

视图的修改

① 创建或修改

修改视图 view-CS, 删除备注字段 remarks

```
CREATE OR REPLACE VIEW view-CS
```

```
AS
```

```
SELECT sno, sname, dept, birthday, totalcredit
```

```
FROM student
```

```
WHERE dept = '计算机';
```

② 修改

修改视图 view-CS, 增加字段 sex

```
ALTER VIEW view-CS
```

```
AS
```

```
SELECT sno, sname, dept, birthday, totalcredit
```

```
FROM student
```

```
WHERE dept = '计算机';
```

视图的更新

- 要使视图可更新, 则视图中行列与底层基本表之间必须存在一对一关系, 行列子集视图可更新
- 视图一般用来简化查询工作, 尽量不用视图做数据更新操作
- 定义视图的SELECT语句后字段列表中使用DISTINCT、聚合函数, 子查询中有GROUP BY、HAVING、UNION短语等, 该视图不支持数据更新

↳ 主要是2类 $\times$ $\left\{ \begin{array}{l} \text{视图数据与基本表的对不上} \\ \text{计算值不是基本表原始数据} \end{array} \right.$

行列子集视图是可更新的, 但也有例外:

- ① 带表达式的视图、分组的视图
- ② 定义视图的SELECT语句中使用DISTINCT
- ③ 视图中未包含基本表中被定义为非空的列

① 通过视图插入数据

使用 INSERT 语句向通信工程学生视图 view-ce 插入一条数据

```
INSERT INTO view-ce  
VALUES ('001208', '李荣', '通信工程', '女', '2004-4-1', 0, NULL);
```

↓更新基本表

```
INSERT INTO student(sno, sname, dept, sex, birthday, totalcredit, remarks)  
VALUES ('001208', '李荣', '通信工程', '女', '2004-4-1', 0, NULL);
```

使用 INSERT 语句向计算机系学生视图 view-cs 插入一条数据。

也可以成功, 因为定义 view-cs 时无 WITH CHECK OPTION 参数

```
INSERT INTO view-cs  
VALUES ('001114', '王红', '通信工程', '男', '2004-4-23', 0, NULL);
```

② 通过视图修改数据

使用 UPDATE 语句将 view-ce 中 sno 为 001201 的学生的 remarks 改为 '优秀团员'

```
UPDATE view-ce  
SET remarks = '优秀团员'  
WHERE sno = '001201';
```

↓更新基本表

```
UPDATE view-ce  
SET remarks = '优秀团员'
```

```
WHERE sno = '001201' AND dept = '通信工程';
```

由于定义时指定 WITH CHECK OPTION 而自动添加

③ 通过视图删除数据

使用 DELETE 语句将 view-ce 中 sno 为 001203 的学生删除

```
DELETE FROM view-ce.  
WHERE sno = '001203';
```

↓更新基本表

```
DELETE FROM student  
WHERE sno = '001203' AND dept = '通信工程';
```

执行失败, 不能删除: 学号 001203 的学生在 score 表中被引用 (是 score 表的外键, 有引用完整性约束)

视图的删除 (只删除视图的定义而不删除视图所依赖的基本表的数据)

删除视图 view-cs-102

```
DROP VIEW view-cs-102;
```

由它导出的视图 view-cs-102-90 仍存在, 但失效。

练习

导入表 zlgc (序号 int, 项目名称 varchar(255), 单位 varchar(255), 项目类别 varchar(255), 负责人 varchar(255), 等级 varchar(255))

① 在“序号”上创建主键索引 primary

```
ALTER TABLE zlgc  
ADD PRIMARY KEY(序号);
```

```
CREATE INDEX idx-pname  
ON zlgc(项目名称);
```

② 在“项目名称”上创建前缀索引 idx-pname, 并判断前缀字符个数

```
CREATE INDEX idx-pname  
ON zlgc(项目名称(10));  
SELECT COUNT(DISTINCT LEFT(项目名称,10))/COUNT(项目名称)  
FROM zlgc;
```

③ 在“负责人”上创建全文索引, 并查找“曹慧平”参与的项目

```
CREATE FULLTEXT INDEX ft-person  
ON zlgc(负责人);  
SELECT * FROM zlgc  
WHERE MATCH(负责人) AGAINST('曹慧平');
```

④ 在“单位”“项目类别”上创建联合索引 idx-school-category

```
CREATE INDEX idx-school-category  
ON zlgc(单位, 项目类别);
```

⑤ 创建视图 v-maxschool 保存哪个学校获得项目最多

```
CREATE VIEW v-maxschool  
AS  
SELECT 单位, COUNT(*) FROM zlgc  
GROUP BY 单位 ORDER BY COUNT(*) DESC LIMIT 1;
```

⑥ 创建视图 v-school-categorys 保存每个学校在不同项目类别上分别获得多少个项目

```
CREATE VIEW v-school-categorys  
AS  
SELECT 单位, 项目类别, COUNT(*) FROM zlgc  
GROUP BY 单位, 项目类别;
```

⑦ 分析以上两视图执行时对索引 idx-school-category 的使用情况

```
EXPLAIN SELECT * FROM v-maxschool;
```

```
EXPLAIN SELECT * FROM v-school-categorys;
```